

Project 4: Wumpus World and Tic-Tac-Toe

Goal. Build two AI agents: (1) a **Wumpus World agent** using **knowledge-based reasoning/logic**, and (2) a **Tic-Tac-Toe agent** using **adversarial search (Minimax and optionally Alpha-Beta pruning)**. You will implement the two systems, run required experiments, and evaluate agent behavior using logged metrics.

1. What You Will Build

You will write a program that:

- Solves **Wumpus World** instances using a **knowledge base** and inference rules.
- Plays **Tic-Tac-Toe** using **Minimax** against a configurable opponent.
- Supports multiple agent **configurations** for comparison.
- Prints results to the terminal **and** saves results to a **JSON file**.

Important constraints:

- Your Wumpus World agent must reason using **explicit facts/rules** (not hard-coded move sequences).
- Your Tic-Tac-Toe agent must implement **Minimax** correctly.
- You may choose to implement **Alpha-Beta pruning** (recommended) as an additional configuration or replacement.
- The two parts of the project are different AI paradigms and should be implemented as **two separate modules** in the same project submission.

2. Wumpus World Requirements

2.1 Environment representation

Your Wumpus World environment must support:

- A grid world.
- A starting square.
- Hidden pits.
- One Wumpus.
- Percepts such as **Breeze** and **Stench**.

2.2 Knowledge representation

Your Wumpus World agent must maintain a knowledge base containing:

- facts obtained from percepts,
- world rules (for example, Breeze implies at least one adjacent pit),
- derived conclusions about safe or unsafe squares.

2.3 Required agent behavior

Your agent must:

- update the KB after receiving each percept,
- infer whether adjacent squares are safe/unsafe/unknown,
- choose moves using its current knowledge,
- avoid moving into squares that are known to be unsafe.

3. Problem A: Wumpus World (Knowledge-Based Agent)

3.1 State/world assumptions

You may assume:

- a fixed grid size,
- deterministic percept rules,
- one Wumpus,
- static hazards.

3.2 Required logical rules

At minimum, your KB should support rules of the form:

- If there is **no breeze** in a square, then adjacent squares contain **no pits**.
- If there is **no stench** in a square, then adjacent squares contain **no Wumpus**.
- If there is a **breeze**, then at least one adjacent square may contain a pit.
- If there is a **stench**, then at least one adjacent square may contain the Wumpus.

3.3 Required Wumpus instances

You must test your agent on **at least 4 Wumpus World layouts**:

- 2 easier layouts where safe moves can be inferred quickly,
- 2 harder layouts where uncertainty remains for longer.

You must include these layouts in your submission.

4. Tic-Tac-Toe Requirements

4.1 Game representation

Your Tic-Tac-Toe module must support:

- a 3×3 board,
- alternating turns,
- terminal tests for win/loss/draw,
- a utility function for terminal states.

4.2 Required search algorithm

You must implement:

- **Minimax** for move selection.

Recommended (not required):

- **Alpha-Beta pruning** as an optimized version of Minimax.

4.3 Required opponents

You must test your Tic-Tac-Toe agent against:

- a **random opponent**,
- a **human player** or a deterministic scripted opponent.

5. Problem B: Tic-Tac-Toe (Adversarial Search)

5.1 Utility values

Use terminal utilities such as:

- Win = +1
- Draw = 0
- Loss = -1

5.2 Required Tic-Tac-Toe runs

You must run:

- **at least 10 games** against a random opponent using Minimax,
- **at least 5 games** against a human or scripted opponent.

If you implement Alpha-Beta pruning, you must include it as an additional comparison configuration or replace Minimax with Alpha-Beta pruning.

6. Algorithms/Configurations to Implement

You must implement and run the following:

6.1 Wumpus World configurations

- **KB baseline:** a knowledge-based agent using your required inference rules.

6.2 Tic-Tac-Toe configurations

- **Minimax:** standard minimax search. or
- **Minimax + Alpha-Beta:** if you implement Minimax and pruning. or
- **Alpha-Beta:** if you implement pruning.

7. Program Inputs and Outputs

7.1 Required inputs (CLI arguments or config file)

Your program must accept:

- **problem:** `wumpus` or `tictactoe`
- **instance:** identifier for the Wumpus layout or Tic-Tac-Toe mode
- **config:** for example, `kb`, `minimax`, or optionally `alphabeta`
- If randomization is used (for layout selection, random opponent, or move order), a **seed**

7.2 Required output for every run

For each run, you must:

- Print results to the terminal, and
- Save the same results to a **JSON file**.

Your JSON output must include at least these fields:

- **problem** (`"wumpus"` or `"tictactoe"`)
- **instance** (string id)
- **config** (`"kb"`, `"minimax"`, `"alphabeta"`, ...)
- **success** (`true/false`) for Wumpus, or **result** (`win/loss/draw`) for Tic-Tac-Toe
- **runtime_ms**
- **states_expanded** or **nodes_evaluated** (where applicable)
- **moves_taken**
- **trace** (sequence of moves / actions / percept updates / board states)

8. AI Usage Policy (Two Options)

You may choose either option. Your grade will be computed using the matching rubric.

Option A: You do not use AI tools for coding

You are graded mainly on:

- correct implementation and design,
- testing, and
- completing the required runs and discussion.

Option B: You use AI tools for coding

You may use AI tools to generate code, but you are graded mainly on:

- experimental design and correctness of results,
- quality of analysis and explanations,
- evidence that you validated the code (tests + sanity checks), and
- an AI disclosure appendix (see below).

AI disclosure appendix (required only if you used AI)

Include in your report:

- which tool(s) you used,
- what you used them for,
- at least 3 example prompts, and
- what you did to verify the code is correct.

9. Testing Requirements (For both AI options)

Include at least:

Wumpus World

- A check that the percept rules are implemented correctly.
- A check that the agent does not move into a square known to be unsafe.
- A check that the knowledge base updates after each percept.

Tic-Tac-Toe

- A validity check that legal moves are generated correctly.
- A validity check that terminal states are recognized correctly.
- A check that Minimax chooses a utility-maximizing move on small test boards.

10. Experiments and Analysis Requirements (For AI Option B)

10.1 Required runs

You must run all required instances under all required configurations:

- 4 Wumpus layouts $\times$ 1 configuration = **4 runs minimum**
- 15 Tic-Tac-Toe games minimum (**10 random + 5 human/scripted**)

If you implement optional configurations (enhanced inference, Alpha-Beta), include them in your comparison.

10.2 Metrics you must report

In your report, compare and discuss:

- **Success/failure** in Wumpus World
- **Win/loss/draw** results in Tic-Tac-Toe
- **Runtime**
- **Moves taken**
- **Nodes evaluated** (for Minimax / Alpha-Beta)

10.3 What analysis is expected

Your report must clearly explain:

- How the Wumpus agent uses percepts to infer safe or unsafe squares.
- At least **one concrete example** of a logical inference made by your Wumpus agent.
- Why Minimax chooses the move it does on at least **one specific Tic-Tac-Toe board**.
- If you implement Alpha-Beta pruning, how pruning reduces node evaluation compared with plain Minimax.

11. Deliverables (What to Submit)

Submit the following on Canvas:

11.1 Code package

A zip file or repository link containing:

- Python source code
- `README.md` with instructions to run (example commands)
- The Wumpus layouts and Tic-Tac-Toe test setups you used
- The JSON outputs from your runs (or a script that regenerates them)

11.2 Report (PDF, 3–6 pages)

Your report must include:

- Your Wumpus World representation (grid, percepts, rules, KB behavior)
- A brief description of your Tic-Tac-Toe implementation (Minimax; optional Alpha-Beta)
- Your testing summary (what you tested)
- Your results table and discussion (**only for AI Option B**)
- AI disclosure appendix (only if you used AI) (**only for AI Option B**)

12. Grading Rubric (100 points)

Option A: No AI for coding (implementation-focused)

- [20 pts] Wumpus World design and correctness (environment + KB + safe action selection)
- [20 pts] Tic-Tac-Toe correctness (game logic + Minimax soundness)
- [15 pts] Knowledge-based reasoning quality (rules implemented correctly; inference behavior reasonable)
- [15 pts] Adversarial search implementation (correct utility propagation; move choice correct)
- [10 pts] Wumpus testing and required layouts
- [10 pts] Tic-Tac-Toe testing and required games
- [5 pts] Metrics + JSON correctness (required fields present and accurate)
- [5 pts] Report quality (clear explanation; includes how to run + what you tested)

Option B: AI used for coding (analysis-focused)

- [15 pts] Problem formulation clarity (Wumpus + Tic-Tac-Toe represented correctly)
- [25 pts] Experimental completeness (all required runs; JSON saved; configurations compared)
- [25 pts] Analysis quality (clear Wumpus inference explanation; Tic-Tac-Toe decision analysis; pruning comparison if applicable)
- [15 pts] Validation evidence (tests + sanity checks; shows you verified correctness)
- [15 pts] Report quality + AI disclosure (prompts, usage, modifications, verification described)
- [5 pts] Implementation soundness (runs correctly; produces valid behavior)

13. Suggestions

- Keep the Wumpus World environment small at first (for example, 4×4) and verify percept generation carefully.
- Print a clear reasoning trace for the Wumpus agent so you can debug KB updates and inferences.
- For Tic-Tac-Toe, test Minimax first on near-terminal boards before running full games.
- If you implement Alpha-Beta pruning, count node evaluations so the comparison is meaningful.